Lecture - 1

Software engineering. [SWE].

→ Software.

- Instructions or computer programs that when executed to provide desired features, functions & performance
- D.S. that enables the program to adequately manipulate the information.
- Documents that describe operations & use of the program.

→ Evolutionary Role of Software.

1. Product →

As a product, it delivers the computing potential embodied by computer hardware or more broadly a network of computers accessible by a local or global hardware.

2. Vehicle & Platform →

As a vehicle, it is used to deliver the product by acting as the basis for the control of computer. (computing), communication of info. & creation & control of other programs

→ Functions provided by Software.

1. It delivers most important products. (info & data)
2. It provides the gateway for world wide info.
3. It provides the means for exchanging info.
4. It manages the business info efficiently

→ Categories of Software.

1. System Software.
2. Realtime Software.
3. Business.
4. Engineering & Scientific.
5. Embedded Software. (inside ROM)
6. PC Software.
7. Web Based.
8. AI Based.

* Software Agreement Instruction.

* CASE :- Computer Aided. Software Engineering

* Compiler. → LEX → Yacc.

Lecture - 2

Software engineering.

✓ According to Bauer, S.W. Eng is defined as establishment of sound eng principles in order to obtain the SW i.e. economical, reliable and coode efficiently on real machines.

✓ According to ~~Boehen~~ Boehen. SW Eng is application of science and mathematics. by coGich the capabilities of comp. equipments are made useful to man via computer programs., procedures and related documentation.

✓ According to IEEE, SW Eng. is the systematic approach to the development, operation, maintenance and retirement of the software. It is the technological and managerial discipline concerned with systematic production and maintenance of software products that are developed and modified on time and within cost estimates.

→ Goals of SW engg.

1. To improve the quality of software.
2. To increase productivity.
3. To increase the job satisfaction of software engineers.

→ Standard Terms.

How much work is done in unit time?

Project size.

It is a major factor that determines the level of management control and the types of tools and techniques required on a software project.

Four categories of projects.

1). Trivial project.

- ✓ It involves 1 programmer often working part time.
- ✓ Time spent - few days / few weeks
- ✓ Program will contain upto 500 LOC.
- ✓ SW would contain 10 to 20 subroutines.
- ✓ They're mostly personal SW. for exclusive use of programmer.
- ✓ Very little need for formal analysis., elaborate design, documentation or extensive planning.

Scanned with OKEN Scanner

2) Small Project

- Involves 1 programmer.
- Duration - 1 to 6 months
- Program can contain 1000 - 2000 LOC, 25 - 50 subroutines / routines.
- They do not usually have interaction b/w other programs.
- Generally used for small commercial application such as report-gen and simple data manipulation
- Negligible interaction b/w customer & programmer.

3). Medium-sized projects.

- 2 - 5 programmers.
- 1 - 2 years.
- 10,000 - 15,000 LOC
- 250 - 1000 routines.
- Includes assemblers, compilers, process-control systems, inventory systems, small MIS softwares etc.
- Requires medium interaction b/w programmers & customers.
- certain formality is reqd. in planning, degree of documentation and project reviews.

4) Large ~~programmers~~ projects.

- 5 - 20 programmers
- 2 - 3 years
- 50 k - 1 Lakh LOC.
- Entire project is a combination of several sub systems.
- The modules in the project have significant interaction. btw other programs and SW systems.
- It requires multi-level management.
- small time sharing systems.
  large compilers.
  graphic software for data acquisition & display.
  real time control systems.
- Projects can involve extensive planning, analysis, risk est., standardised documentation, formal reviews etc.

5) very large projects.

- 100 - 1000 programmers.
- 4 - 5 years.
- 10 lakh LOC.
- project is divided into several major subsystems with extensive interactions.
- subsystems are extremely complex & developed separately under expert guidance.

Scanned with OKEN Scanner

- These projects require real time processing, telecommunication and multi tasking.
- Ex. include large OS, military command and control systems, mega database sys.

6) Extremely large projects.
- 2k - 5k programmers.
- nearly 10 years.
- 1 million - 10 million LOC
- entire project is an integration of several extremely large systems. which further consists of several large subsystems.
- Projects involve real time processing, telecommunication, multi tasking and distributed system.
- Ex: air traffic control sys.
  ballistic missile control sys.
  [Hard RTS].
  millitary command & control sys.

Scanned with OKEN Scanner

Lecture. 3.

Date 19/02/25
Page

→ Verification and validation.

Goal: asses and improve the quality of the work products generated during development and modification of software.

Qualitative attributes:

1. Correctness.
2. Completeness. (based on the specificity)
3. Consistency.
4. Reliability (& Robustness) → Contingency plan!
5. Usefulness.
6. Conformance to Standards.
7. Overall cost effectiveness.

Assessment of work products to determine conformance to the specifications.

Specifications include:

1. Requirements specification.
2. Design documentation.
3. Various stylistic guidance
4. Implementation language standards.
5. Project standards.
6. Organizational standards.
7. User expectations.

Scanned with OKEN Scanner

→ Verification.

↳ done to ensure that the work product of a given phase of the development cycle fulfil the specifications established during prior phase.

According to Boehm, Verification:
Are we producing the product right?

Verification ensures that the product is designed to deliver all functionality to the customer.

It typically involves reviews and meetings to evaluate documents, plans, code, requirements and specifications.

This can be done with checklist, issue lists, walkthroughs and inspection meetings.

In verification, uncovering of defects will be done in primary ways.

Here, no code will be executed.

Before building actual system this checking will be done. This process will be called Quality Assurance.

Verification is nothing but the Static Testing

Inputs : check list, issue list, walkthroughs and inspection

→ Validation.

↳ process of evaluating software at the end of the software development process to determine compliance with requirements.

" Are we producing the right product ? "

Validation ensures that the functionality as def. in req. is the intended behaviour of product.

Involves actual testing.

Takes place after ver. is completed.

Validation concern - Quality Control

" Dynamic Testing "

Input :

Output :

Lecture. 4

19/08/26.

→ Software Development Life Cycle.

1. Customer communication.
2. Requirement gathering & analysis.
   BRD is prepared.
   [Business Requirement Document].

Types of Models.

1. Waterfall model.
2. Incremental model.
3. Spiral model.
4. WINWIN Spiral model.
5. RAD model.
6. Prototyping model.
7. Object - oriented model.

→ Waterfall Model.
[Linear Sequential Model].

Lecture. 5

~~Waterfall~~.

21/08/26.

→ Iterative Model.
→ Spiral Model.
→ WINWIN Spiral Model.
→ RAD Model.
→ Prototyping Model.
→ object oriented Model.

Scanned with OKEN Scanner

"How well do I know the requirements, and how much can they change?"

Fixed. Fully known upfront, delivered as one single block → Waterfall.

Tech feasibility itself is unknown (regardless of cost) → Prototyping.

Uncertain / High risk, large-scale or costly project → Spiral.

Uncertain / High risk + Multiple stakeholders with conflicting interests → WINWIN Spiral

Known in outline, but will grow / can't wait for full delivery → Incremental.

Well understood + tight deadline (60-90 days) → RAD.

Reusable objects / components exist → Object-Oriented (component Assembly).

Waterfall.

Do it once, do it in order - and deliver it whole.

Best For: Stable requirements delivered as ONE release - simple projects (data processing) but also larger ones. (eg peripheral device inventory sys.) as long as nothing needs to ship incrementally.

Incremental.

Ship it in slices.

Best For: long projects, limited staff, need working software early (eg. Inventory MIS).

Spiral.

Loop through risk.

Best For: large, expensive, high-risk / mission-critical sys. (eg. fingerprint matcher).

WINWIN Spiral.

Everyone must WIN before we spiral.

Best For: Spiral + multiple stakeholders who must agree on requirements.

BAD.

Reuse + Speed.

Best For: Well-understood, modular info-systems needing fast delivery

Scanned with OKEN Scanner

Prototyping:

Build a throw away to learn

Best for: Requirements unclear or tech feasibility unknown (eg satellite sys.)

Object - Oriented.

Build with lego Blocks.

Best for: Component-Based / Reusable - library - driven development.

Scanned with OKEN Scanner

Is it unclear. coetur. current tech can acheive the goal ?

[YES.] → PROTYPING.
(regardless of cost - test feasibility first).

[NO] → Is. scope / requirements fixed . AND delivered as. ONE block ?

[YES]. → WATERFALL.

[NO]. → Is it high- risk / very costly / large scale ?

[YES] → Multiple stakeholders. with conflicting needs. ?

[YES]. → WINWIN Spiral.

[NO]. → Spiral.

[NO]. → Need it fast- (60-90 days) with. reusable parts ?

[YES] → RAD.

[NO]. → Incremental

Model.	Advantages	Disadvantages
Waterfall.	Simple. Systematic. time-tested.	Rigid - real projects rarely stay linear.
Incremental.	Working SW early, flexible scope.	Time-consuming architecture; architecture issues if not all requirements gathered upfront.
Spiral.	Great for large/ risky systems.	Needs strong risk analysis expertise; costly.
WINWIN Spiral.	Builds trust among stakeholders	Still inherits spiral's cost/complexity.
RAD.	Fast overall development via reuse	Fails without full team + Customer commitment; bad for high technical risk.
Prototyping.	Great for nailing down requirements.	Customer may mistake prototype for final product; time-consuming.
Object-Oriented.	Reuses existing components.	Needs a mature component library to be effective.

Lecture: 6

04/07/24.

(3).

—> Agile Model.

It was primarily designed to help a project to adapt to change requests quickly.
The main aim of agile model is to facilitate quick project completion.

This is acheived using task agility.
Agility is acheived by fitting the process to the project, removing activities that may not be essential for a specific project.
Also anything that is wastage of time and efforts is avoided.

Agile model refers to a group of development processes. These processes share some basic characteristics but do have certain subtle differences among themselves.

Some Agile Models are as follows :

1. Crystal.
2. Atern.
3. Feature - driven development
4. Scrum.
5. Extreme programming (XP).
6. Lean development.
7. Unified process.

"Most used. Models"

Scanned with OKEN Scanner

Agile Model is the combination of iterative and incremental processes models.

Phases of Agile Model :

1. Requirement Gathering
2. Req. Analysis.
3. Design.
4. Coding
5. Unit Testing.
6. Acceptance Testing.

The time to complete an iteration is known as a "Timebox".

Timebox refers to the maximum amt. of time needed to deliver an iteration to customer. Thus the end date for an iteration does not change.

The central principle of agile model is the delivery of an increment to the customer after each timebox.

Scanned with OKEN Scanner

iterative

known

of time

to

an

the

customer

→ D. Principles of Agile Model.

1. To establish close contact with the customer during development.
   Agile model usually includes a customer representative in the team. At the end of each iterations, stakeholders & customers rep. review progress of project to re-evaluate the requirements.
   ↳ D. clear understanding of req.

2. Agile model relies on working software development rather than comprehensive documentation.

3. Frequent delivery of incremental versions of the software to the customer representative in intervals of few weeks.

4. The changed requests from the customer are efficiently incorporated.

5. It emphasizes on efficient team members and their communication through face to face models rather than exchange of formal documentation.

6. It encourages the use of peir programming.
   In peir programming, two programmers work together at one workstation where one does coding and other reviews the code while it is being typed.

→ Disadv. of Agile Models.

1. Lack of formal documentation.

(7).

→ Formal Specification based Model.

It is only for scientific projects & applications.

Since mathematical approach is employed, clear specifications are available.

Since clear specifications are available, verification becomes easier. Since verification is easy, it results in defect-free product availability.

Formal methods / Formal model / Specification-driven model. / Transformational process Model.

(11)

(Concurrent process Model)

→ Client - Server Model.

Client - Server

Parallel system.
⇓
centralised.
⇓
Tightly coupled system.
⇓
Maximum sharing of resources.
⇓
sparse network.

Distributed system.
⇓
decentralized.
⇓
loosely coupled system.
⇓
Minimum sharing of resources.
⇓
server has "KNOW - HOW" parameter - s
(token).
⇓
Denser network.
(multiple requests)

→ Summary of SDLC.

1. Waterfall Model.
2. Component based Model.
3. Formal Model.
4. Evolutionary Models [Iterative Models].
   a) Spiral.
   b) WINWIN Spiral. (only 1 prototype is made)
   c) Prototype.
      i. Evolutionary prototype.
      ii. RAD. (rapid app. dev.)
      Based on LOC. [Organic / Semi detached / Embedded. Software projects]
      Used only for organic projects.
      iii. Throw away prototype.
      (n - prototypes are created. no storing)
5. Incremental Models.
   multiple versions of a single project is created.

(5)

→ Process Model.

Purpose of process model :

1. To describe the process.
2. Steps / Guidlines related to development
3. Modification of software.

Microsoft → Stabilize / synchronise.
IBM → Rationalised unified process model.
Infosys. → Waterfall / Spiral.

Aims of Process Model :

1. Acheive quality
2. continuous improvement
3. control process activities.
4. Estimation of efforts for a product development.

For eg. :- IBM uses inception, elaboration, construction, transition.

31/08/26.

Unit - 2.

31/08/26.

Software Requirement Specification.

——>. Semantic versioning.

Build. version. Release.

Version ——> changes in documentation.
+ software.

Scanned with OKEN Scanner

02/09/26

02/09/26

Unified Modelling Language (UML)

platforms :-

a) diagram.io
b) Agro UML.
c) Star UML

What are UML?

These are diagrams. UML is a standard visual language used to model, visualize & document a software system. Instead of explaining a S/W sys through text or code only, UML allows us to rep. it visually.

For example :-

a student can register for a campus event.

Fundamentally, it does not tell us :

a) Who interacts with the system.
b) What exactly happens
c) What info. is involved.
d) Which objects communicate.
e) What happens if registration is full.

UML allows us to represent these aspects using different diagrams.

Scanned with OKEN Scanner

→ UML Diagram selection.

Model.	Question.
1) Scenario.	What happens in a particular situation.
2) Use case.	Who uses the system. and what do they want.
3) Activity.	What is the work flow.
4) Class Diagram.	What concepts or classes. exist.
5) object Diagram.	What actual instances. exist.
6) Collaboration.	Which objects communicate.
7) State Diagram.	How does an object change.
8) Component Diagram.	What software write. make up the system.

Scanned with OKEN Scanner

For Example.

In case of CampusHub, it allows students to register for university events.

Scenario v/s. Activity.

Class. v/s. Object.

Note:

UML is not the software itself rather is a visual model of the software itself.

Different UML diagrams provide different views of the same system.

Scanned with OKEN Scanner

→ Scenario Diagram.

A scenario is a specific sequence of events or interactions that describe one particular situation in the system.

For example,
an FR says - Register for event
It means a student wants to attend a technical workshop and registers for it through campus Hub.

If requirement is :-
Student should be able to register for an event.

Scenario :-
student selects an available event, clicks register, the system validates the request, creates the registration and displays confirmation

Note :-
Requirement is a general statement of need. While a scenario is a specific sequence of interactions.

All of this is a BOT.

Scenario :

Successful Event registration.

Actor :

Student. Be specific!

Goal :

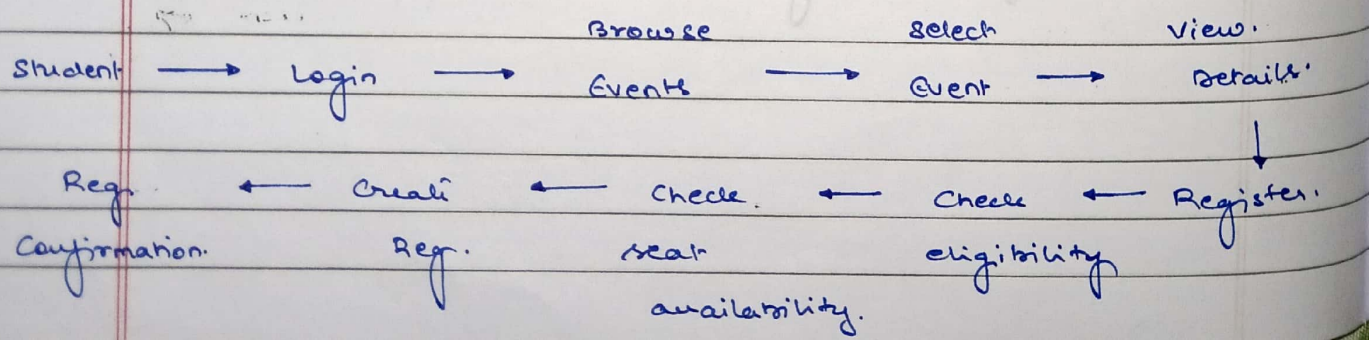
Successful registration for Campus Event.

OR. Registration. for Event.

Main Flow :

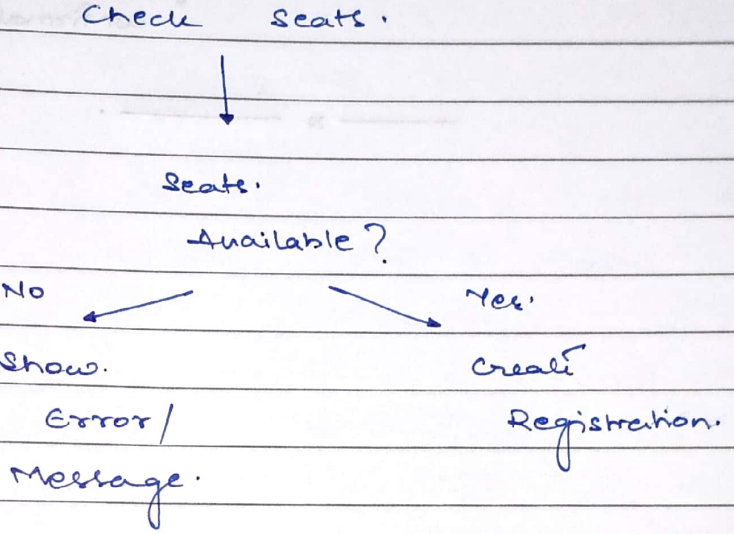
1. Student logs in to Campus Hub
2. Campus Hub displays the available events.
3. Student selects an event
4. Campus Hub displays event details.
5. Student selects. "register".
6. Campus Hub checks whether registration is open.
7. Campus Hub checks whether student is eligible.
8. Campus Hub checks whether seats are available.
9. Campus Hub creates the registration.
10. Campus Hub ~~creates~~ a registration confirmation displays.

Representation :



Alternatives :-

1. Event is full.
2. Reg is Closed.
3. Student already registered. (S).



Scenario Templati :-

	Field.	Description.
1.	Scenario ID.	SC-01.
2.	Scenario Name.	Succesful Event Reg.
3.	Primary Actor.	Student.
4.	Goal.	Registration for Event.
6.	Mainflow	* * * Next page.
5.	Pre condition.	
7.	Alternative flow.	
8.	Post Condition.	Student reg suc. & confirmation received.

Precondition → Student has logged in.
Event Exists. Reg. is open.
Student has valid credentials.

Mainflow → Student completes registration successfully and gets confirmation.

---x---.

Assignment.

FR : Student is able to cancel an existing event registration.

Scanned with OKEN Scanner

eg. Student
↳ Login
↳ Register for event.

event approver
↳ check & approve event.

Coordinator
↳ create event
↳ certificate generation
↳ add - - .

- A USD provides a HL-view (High-level) of actors who interact with the system.

Use Cases :-
What they want to acheive?

Relationships :-
How actors and use cases are connected?

System Boundary :-
What belongs to that particular system.

Scanned with OKEN Scanner

Date
Page

07/08/26

→ Use Case Diagram.

A use case diagram describes functionality of the sys. that provides a meaningful result to an actor. In simple terms, a use-case is something a user wants to accomplish using the system

Identify a use case.

Relationships are of 2 types:

a) << extend >> (optional)

b). << include >> (mandatory)

→ 5 Questions.

[Make use case Diagram].

1. What is the system? → Rep. using the system boundary
2. Who uses the system? → Identify the actors.
3. What does each actor want to accomplish.? → Identification of use cases.
4. Who performs which use-case ? → Draw the association.
5. Is some behaviour mandatory or optional ?

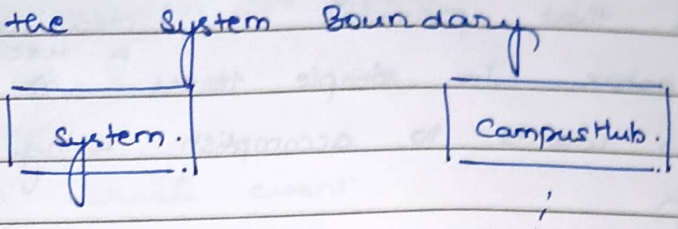
Relationships → Association

<< extend >> << include >>

Scanned with OKEN Scanner

→ UML Representation.

1. Start with the System Boundary



eg.

2. Identify the actors.
   a) Student.
   b) Event Coordinator.
   c) Event Approver.
   d) Administrator.

Stick figure. Actors on the left.

3. Identify the use Case. & Write the actors.

You can write use cases in 2 ways.

① Rounded Rectangle.

② Oval.

4. Association.

An Association shows that an actor participates in a use case.

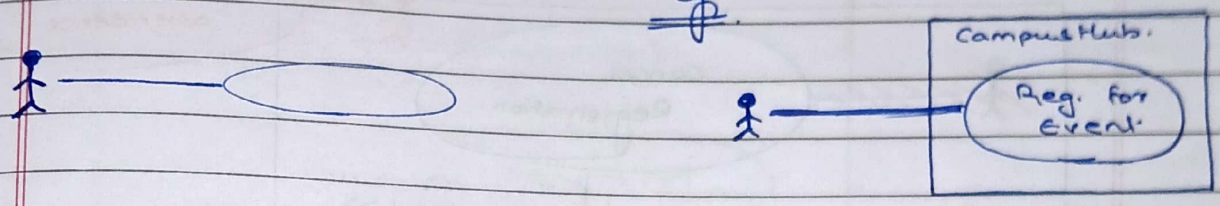
No arrow (only line).

For eg: Student interacts with the "Register for Event" functionality.

Rep. of a use case and actor with a line. External Entity (Student) interacts with the Internal Entity (System's Functionality)

Scanned with OKEN Scanner

eg.

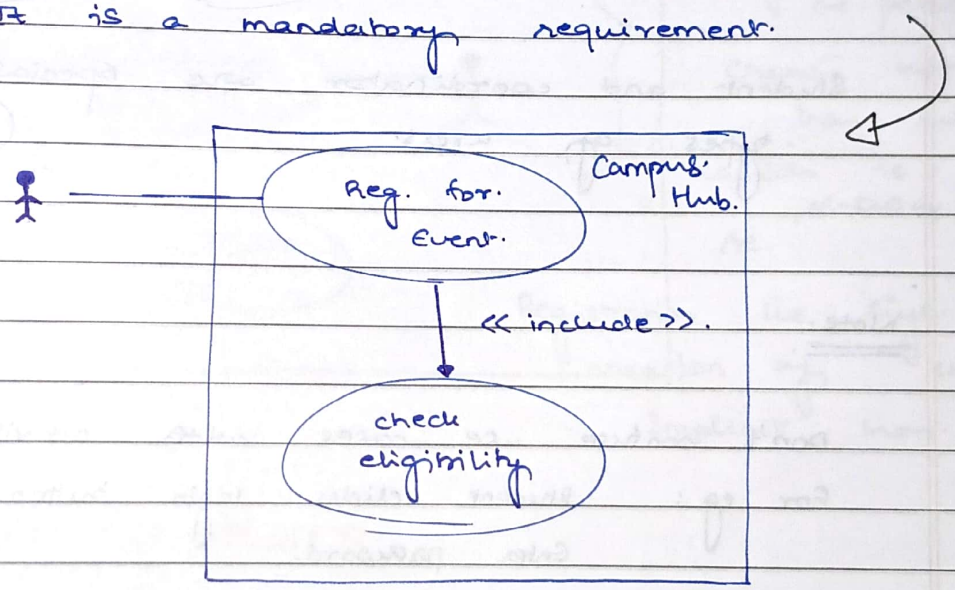


<< include >>.

Suppose every registration requires the system to check availability or validate eligibility.

Use with arrow. [Shows the implicit behaviour]

∴ It is a mandatory requirement.

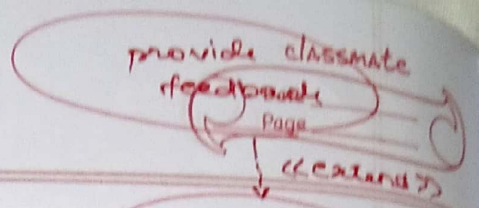


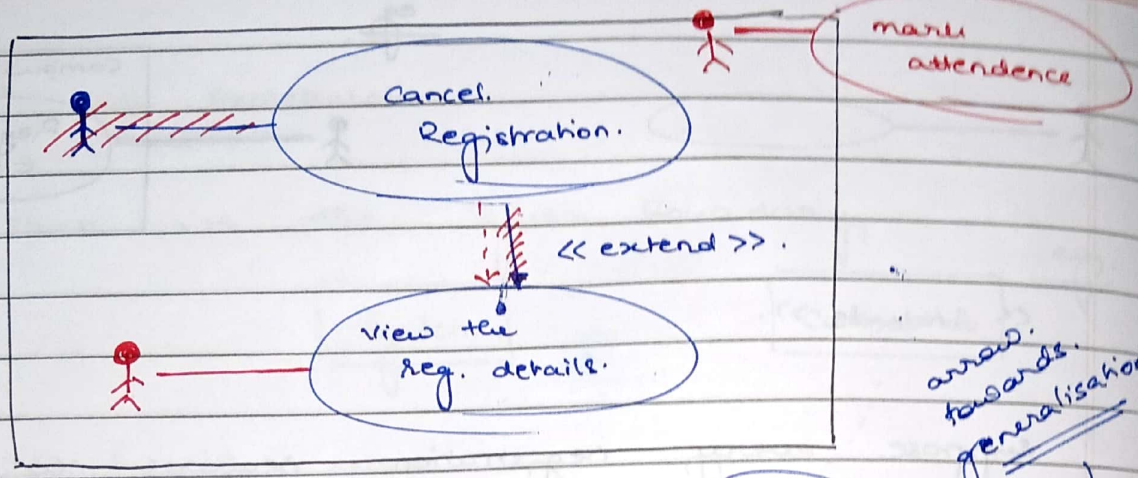
<< extend >>

arrow present.

Consider that the student may cancel registration.

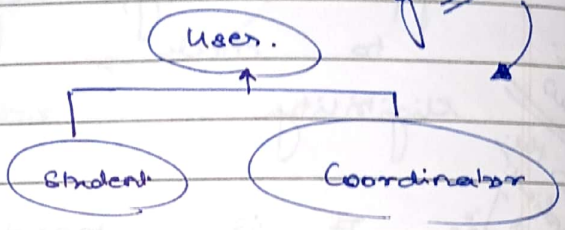
∴ It is an optional requirement OR a different requirement.





arrow towards generalisation.

Generalisation.



Student and coordinator are specialised types of users. (specialisation)

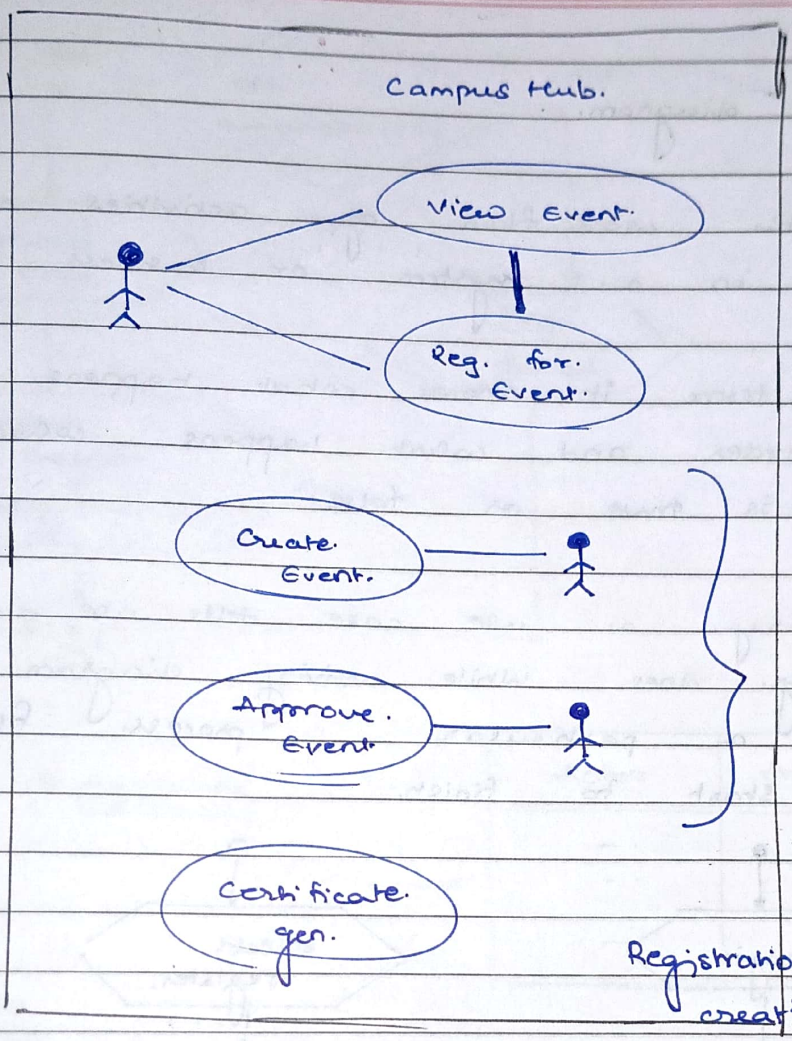
Note.

Don't confuse use cases with activities.

For eg: Student clicks login button.
Enter password.
Enter submit
See / View dashboard.

Inclusion and Exclusion are drawn outside the system boundary.

Scanned with OKEN Scanner



Registration. ke pehle. Create hota hai event. which is shown ealag se.

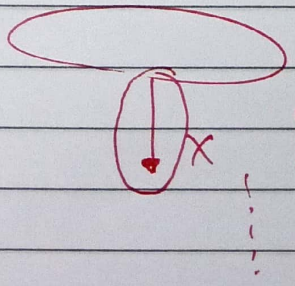
Registration ke liye. creation of event- Implicit hai.

Note.

<<extend>> / <<include>>.

Include means the arrow to which is included i.e. it points to included use case.

Extend. ~~Extends~~ means arrow to which is extended ie. it points to the base use case.



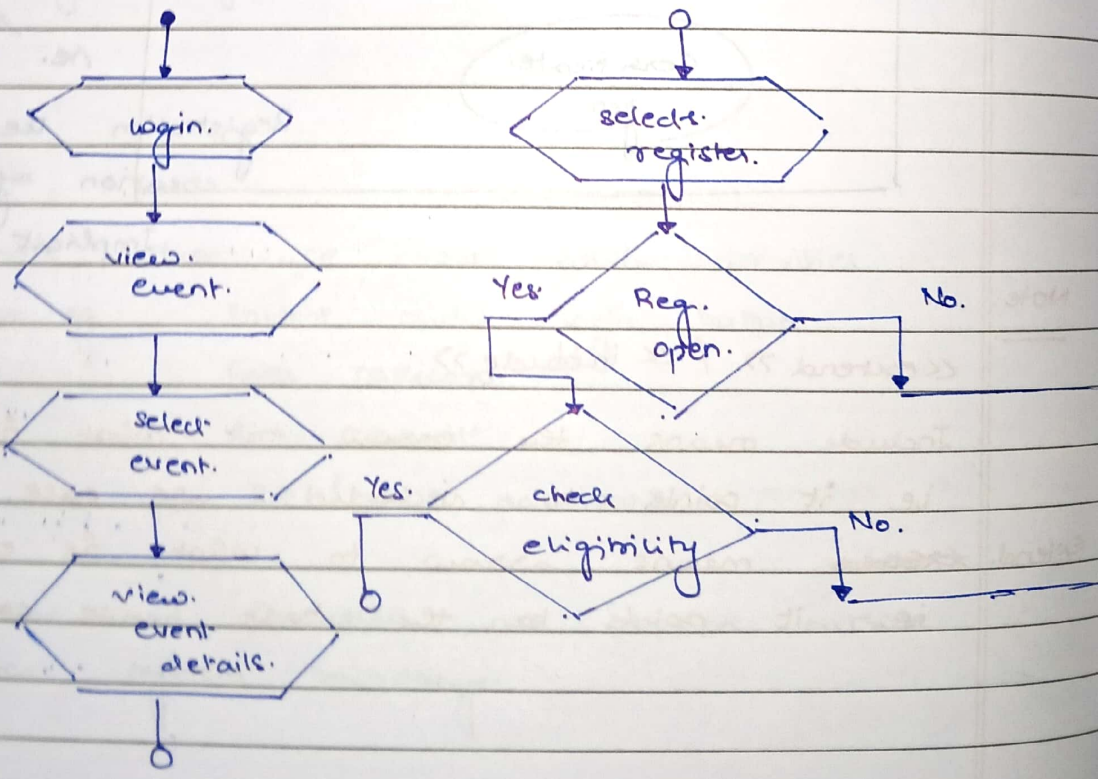
Scanned with OKEN Scanner

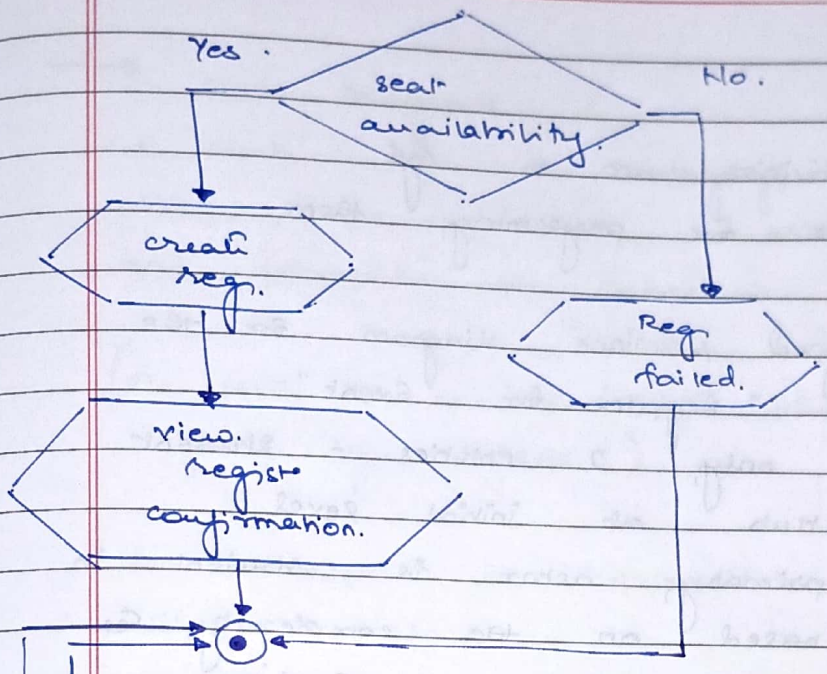
→ Activity diagram.

It rep. the work flow of activities or actions in a system or business process.

In simple terms, it shows what happens in what order and what happens when a condition is true or false.

In summary, a use case tells use what the sys. does. While activity diagram tells us how a particular process flows from start to finish.





ek activity
eg.
dusri activity se nahi jod. sakte.
[No hone par. you can't go to. view events].

All the Yes. are on one side. and all No. on the other side.

Swimlanes.

Actor	Activity.
- .	- .
- .	- .
- .	-

eg.

Student	CampusHub.
login. →	validati login.
view event →	display event
select event →	check availability.
	↓
	!
received. ←	generate conf.
↓	

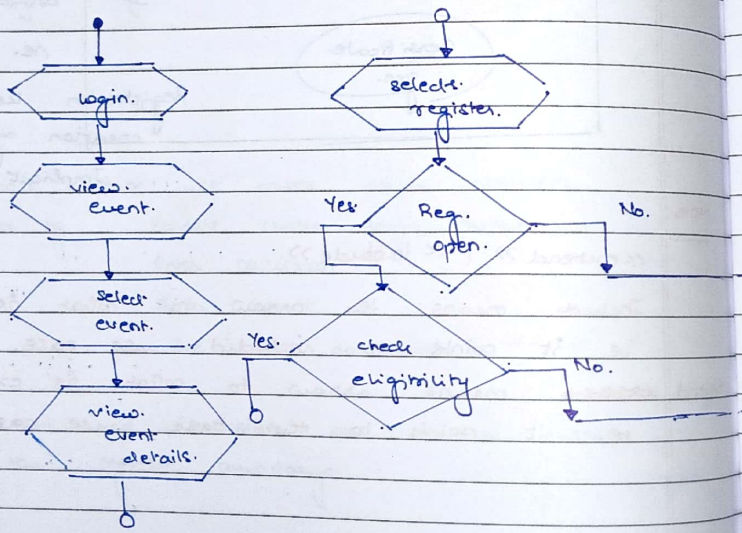
The responsibility is fixed. by the swimlanes.

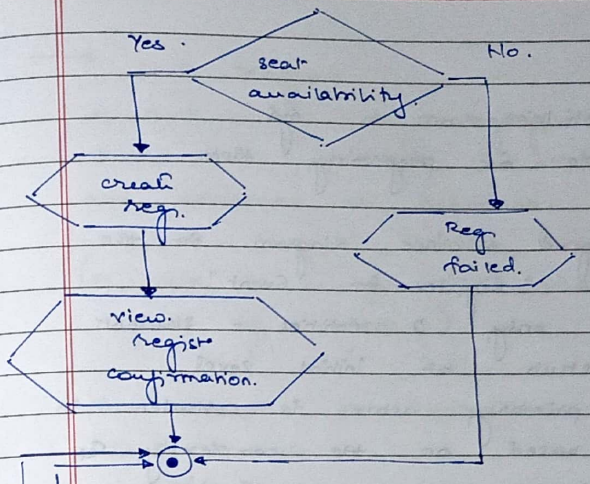
→ Activity diagram.

It rep. the coordi flow of activities or actions in a system or business process.

In simple terms, it shows what happens in what order and what happens when a condition is true or false.

In summary, a use case tells us what the sys. does. While activity diagram tells us how a particular process flows from start to finish.





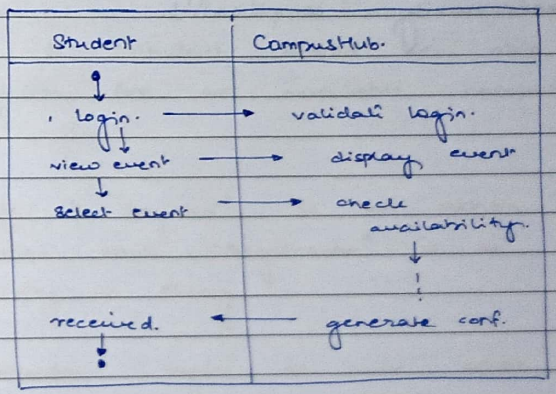
ex activity
ko.
dusri activity ke navi
jod. sakte.
[No hone par. you can't go to. view events].

All the Yes. are on one side and all No. on the other side.

Swimlanes.

Actor	Activity
- .	- .
- .	- .
-	-

eg.



The responsibility is fixed. by the swimlanes.

Scanned with OKEN Scanner

09/08/26

Date
Page

09/08/26.

→ Swimlanes.

It groups activities acc. to sys. responsible for performing them.

For eg: A good swimlane diagram for the use case "Register for Event" should use only 2 entities - Student & CampusHub at initial level. Since the primary actor is student in this case, based on the complexity & the depth of your understanding one can later introduce 2 more entities - namely, event coordinator & event approver. However, to make your swimlane, to specify the usecase, do not make the first diagram unnecessarily complicated.

→ Class diagram.

A class rep. a category or type of objects that share common attributes or data and operations or behaviours.

(A class is a blueprint that specifies a group of objects.)

For example, in Campus Hub :

1. Student class.
2. Event
3. Registration. (registers is not a method ?)
4. Coordinator.
5. Notification.
6. Certificate

Identification of a class from requirement.

eg. R → A student should be able to register for an available campus event.

Candidate class / Identification.

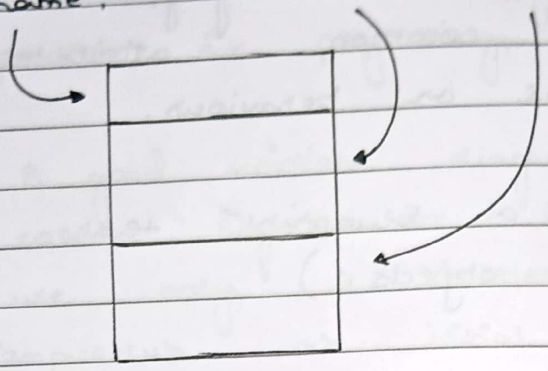
Note :- "Register" is an action or operation. While "Registration" is an information entity or thing.

~~Signup~~ "Login" is an action. It ~~is~~ is an activity.

methods. eg. Execute Functions. Data Page.

Notation of a class.

Class. name. attributes. operations. (methods)



Choose the att. which are important.

eg.

STUDENT.
Student ID
Name
Email
login()
viewevents()
register()

How much atomicity / "complexity" is actually needed ?

REGISTRATION.
Reg ID.
Date.
Status.
confirmation()
Cancel()

EVENT.
Event ID.
Name.
Date.
Venue.
Capacity.
Status.
creatiEvent()
Modify()
publish Event()

creati & publish is different.

why use. Init Capital for nomenclature.

Scanned with OKEN Scanner

Call by value.
—— add.
—— rep

How do you identify an entity as a class?

1. It has information / data.
2. It has its own identity.
3. It has behaviour / responsibilities.
4. It has relationships with other concepts.

——→ OBJECT DIAGRAMS.

An object is a specific instance of a class.

An object is normally rep. using a rectangle where a class diagram shows a general structure. an object diagram shows a specific situation.